정규교과 4/4

머신러닝 모델링 및 성능평가

Day 8 · 평가지표 · 특성 설계 → K-Means → 최종 Pipeline
2026.10.28 (수) | 교차검증 · 최종 평가 · 과목 실습평가

한로 강민준

REVIEW

어제 우리가 배운 것

모델: Decision Tree·Random Forest·XGBoost의 학습 방식을 비교했다.

검증: train 내부 5-fold에서 전처리와 모델을 함께 학습했다.

해석: 중요도·오류·점수 변동을 기록하고 test 20건은 보관했다.

오늘은 지표·특성·모델·임계값을 확정하고, 최종 평가를 재현 가능한 프로젝트로 남긴다.

공식 참고: Cross-validation / Pipelines and composition

한로 강민준

3분 확인 퀴즈 · Day7 복습

Q1. 교차검증 전에 전체 train을 전처리해도 될까?

정답 · 해설 아니다. 각 fold의 학습 부분에서만 대표값·스케일·모델을 fit한다.

Q2. 특성 중요도 1위는 실패의 원인인가?

정답 · 해설 아니다. 특정 모델과 자료에서 사용된 정도이지 인과관계의 증거가 아니다.

Q3. 보관한 test는 언제 사용할까?

정답 · 해설 특성·모델·임계값 등 선택을 끝내고 최종 평가할 때 사용한다.

공식 참고: Cross-validation / Common pitfalls

한로 강민준

오늘의 목표와 8교시 운영

교시	핵심 내용	설명·시연	실습·리뷰
1	Accuracy·Precision·Recall·F1	30분	20분
2	ROC-AUC·임계값·오류 비용	30분	20분
3	RMSE·교차검증·점수 집계	25분	25분
4	Feature Engineering·Pipeline	25분	25분
5	K-Means·군집 평가와 해석	25분	25분
6	선택 확정·재학습·최종 평가	25분	25분
7	재현 가능한 프로젝트 통합	25분	25분
8	과목 실습평가·제출·만족도	15분	35분

50분 × 8교시 = 400분 · 설명/시연 200분 + 실습/리뷰 200분 · 교시 사이 10분 휴식

오늘 남길 세 가지 결과물

평가 기록: 어떤 지표를 왜 골랐는지, 실제값·예측값으로 설명한다.

모델링 프로젝트: 특성 변환·전처리·모델·입력 검사를 연결한다.

최종 보고서: CV와 test 결과, 군집 해석, 재현 방법과 한계를 남긴다.

성능이 낮아도 올바르게 재현하고 한계를 설명하면 평가 기준을 충족할 수 있다.

한로 강민준

개발 데이터와 최종 평가의 역할

train 60

5-fold 비교
전처리도 fold 안에서

valid 20

개발 중 진단
최종 독립 점수 아님

설정 확정

새 Pipeline을
개발80건으로 재학습

test 20

고정 버전 평가
보고 후 재선택 금지

기본 경로: 실패 클래스 F1로 선택 · 임계값은 처음부터 0.5로 고정 · test는 선택에 사용하지 않는다.

공식 참고: Cross-validation / Common pitfalls

전날 환경 유지 · 아직 오늘은 GPU가 필요 없다

```
python --version
python -m pip show scikit-learn xgboost
python -m pip show numpy pandas matplotlib
python -c "import sys; print(sys.executable)"

# 기존 .venv를 VS Code 인터프리터로 선택
# 설치가 필요한 경우에만 강사 검증 환경에 맞춤
# 최종 제출에는 실제 설치 버전을 기록
```

검증 환경

Python 3.13.5
NumPy 2.3.5
pandas 2.2.3
scikit-learn 1.8.0
XGBoost 3.1.3
Matplotlib 3.10.8

이 버전에서 예제를 실행했다.
다른 버전은 별도 확인한다.

공식 참고: Pipelines and composition

공통 데이터 · 예측 시점은 바꾸지 않는다

구분	사용할 내용	이번 수업의 원칙
원본	수업용 합성 요청 100건	Day5~7 원본·ID·분할 순서 재사용
입력 X	글자 수·허용 도구 수·작업 종류·우선순위	요청 실행 전에 확인 가능한 4개 열
분류 y	failed: 성공0 / 실패1	실패1을 양성으로 고정
회귀 y	elapsed_ms: 실행시간(ms)	회귀 지표·교차검증은 같은 ID 사용
금지 입력	정답·status·retry_count·request_id	실행 후 정보와 식별자를 입력에서 제외

합성100건의 성능을 실제 서비스 성능으로 일반화하지 않는다.

공식 참고: Common pitfalls

한로 강민준

좋은 모델의 기준을 먼저 정한다

문제: 무엇을 예측하고, 어떤 실수가 더 큰 손실을 만드는가?

기준: 주 지표 하나를 먼저 정하고 보조 지표와 실제 건수를 함께 본다.

판단: 같은 자료·같은 양성 라벨·같은 기준에서만 모델을 비교한다.

실패 요청을 놓치는 문제와 불필요한 경고를 내는 문제는 서로 다른 오류다.

공식 참고: Metrics and scoring / precision_score / recall_score

한로 강민준

혼동행렬 · 양성은 “실패1”이다

	예측 0 · 성공	예측 1 · 실패
실제 0 성공	TN · 정상 통과 **10**	FP · 오탐 **4**
실제 1 실패	FN · 놓침 **4**	TP · 실패 발견 **2**

읽는 순서

실제 실패 6건 중
2건을 찾고 4건을 놓쳤다.

실패라고 경고한 6건 중
4건은 실제 성공이었다.

설명용 자료 총20건

confusion_matrix(..., labels=[0,1]).ravel() → TN, FP, FN, TP 순서

공식 참고: confusion_matrix

한로 강민준

혼동행렬 · 양성은 “실패1”이다

		실제 정답	
		True	False
분류 결과	True	True Positive	False Positive
	False	False Negative	True Negative

한로 강민준

Accuracy · 전체 정답 비율

정의와 손계산

Accuracy = (TP + TN) / 전체 수

예: (2 + 10) / 20 = 0.600

모든 예측 중 얼마나 맞혔는지 본다.

반드시 함께 확인할 것

실제 양성의 개수와 비율

양성을 전부 놓쳐도
다수 클래스가 많으면 높게 보일 수 있다.

"60% 정확도"와 "실패60% 발견"은 다르다.

전체 정답 비율과 관심 클래스의 발견 비율을 구분한다.

공식 참고: Metrics and scoring

한로 강민준

Precision과 Recall · 분모가 다르다

Precision · 경고의 신뢰도

실패라고 예측한 것 중 실제 실패

TP / (TP + FP)

2 / (2 + 4) = 0.333

오탐이 늘면 낮아진다.

Recall · 실패를 찾은 비율

실제 실패 중 찾아낸 것

TP / (TP + FN)

2 / (2 + 4) = 0.333

놓침이 늘면 낮아진다.

같은 값이 나와도 의미는 다르다. 경고6건과 실제 실패6건은 다른 집합이다.

공식 참고: precision_score / recall_score

F1 · 두 비율의 균형을 한 값으로

정의와 계산

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

$$= 2TP / (2TP + FP + FN)$$

예: $4 / (4 + 4 + 4) = 0.333$

해석과 주의

산술평균이 아니라 조화평균이다.
한쪽이 매우 낮으면 함께 낮아진다.

TN을 직접 반영하지 않으며
업무 비용 전체를 대신하지 않는다.

양성 라벨·분모가 0인 정책도 기록한다.

이 과정의 기본 주 지표는 실패1의 F1. zero_division=0은 계산 불가 상황의 기록 규칙이다.

공식 참고: f1_score

한로 강민준

혼동행렬과 네 지표를 코드로 계산

```
from sklearn.metrics import (confusion_matrix,
    accuracy_score, precision_score, recall_score,
    f1_score)
cm = confusion_matrix(y, pred, labels=[0, 1])
print(cm)
print(accuracy_score(y, pred))
print(precision_score(y, pred, zero_division=0))
print(recall_score(y, pred, zero_division=0))
print(f1_score(y, pred, zero_division=0))
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: confusion_matrix / f1_score

함수 · 매개변수

labels=[0,1]
행·열 순서를 고정
zero_division=0
분모0이면 0으로 기록
기본 양성 라벨: 1

실행 결과

```
[[10  4]
 [ 4  2]]
Acc 0.600
P / R / F1: 0.333
```


한로 강민준

정확도 0.70인데 실패는 0건 발견

```
y = np.array([0]*14 + [1]*6)
pred = np.zeros(20, dtype=int)
print(accuracy_score(y, pred))
print(recall_score(y, pred, zero_division=0))
print(f1_score(y, pred, zero_division=0))
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Metrics and scoring / recall_score

함수 · 매개변수

np.zeros(20)
모든 예측을 성공0으로
accuracy_score
전체 정답 비율
recall_score
실패 중 발견한 비율

실행 결과

Accuracy: 0.70
Recall: 0.00
F1: 0.00
실패 발견: 0/6

지표 선택 이유를 업무 문장으로 쓴다

상황	우선 검토	같이 확인할 것
실패 요청을 놓치면 손실이 크다	Recall 또는 재현율 제약	오탐 수·처리 인력·Precision
경고마다 사람의 확인 비용이 든다	Precision 또는 경고 비용	놓친 실제 실패 수·Recall
두 오류를 함께 줄이고 싶다	F1	FP·FN 건수와 실제 비용
연속적인 실행시간을 예측한다	MAE·RMSE	단위(ms)·큰 잔차·자료 범위

지표 선택은 정답 하나를 찾는 문제가 아니라, 실패의 비용과 운영 조건을 명시하는 일이다.

공식 참고: Metrics and scoring / precision_score / recall_score

한로 강민준

3분 퀴즈 · 분모와 양성 라벨

Q1. Recall의 분모는 “예측한 실패 수”인가?

정답 · 해설 아니다. 실제 실패 수 TP+FN이다. 예측한 실패 수는 Precision의 분모다.

Q2. F1이 높으면 모든 업무 비용도 최소인가?

정답 · 해설 아니다. F1은 오류 비용과 TN을 직접 반영하지 않는다.

Q3. 양성 예측이 없어 Precision을 0으로 기록하면 끝인가?

정답 · 해설 아니다. 양성 예측0건이라는 상황과 zero_division 규칙을 함께 적는다.

공식 참고: precision_score / recall_score / f1_score

한로 강민준

실습 01 · 분모를 계산하고 오류를 설명한다

기본필수

E01의 혼동행렬을 손으로 읽고 Accuracy·Precision·Recall·F1을 계산한다. 양성 라벨과 실제 실패6건을 표시한다.

응용

전부 성공으로 예측하는 E02와 비교한다. 정확도0.70이지만 실패를 못 찾는 이유를 업무 문장으로 적는다.

심화

오탐·놓침 비용이 서로 다른 두 상황을 만들고 주 지표와 보조 지표를 정한다. 분모0 정책도 기록한다.

완료 기준: 혼동행렬 + 네 지표 + 양성/표본 수 + 주 지표 선택 이유

점수·확률·클래스를 구분한다

모델이 출력한 값

predict_proba: 클래스별 추정 확률

실패1의 열을 classes_로 확인한다.

ROC-AUC에는 이 연속 점수를 넣는다.
0/1 최종 클래스만 넣지 않는다.

운영에서 내릴 판단

확률이 임계값을 넘으면 실패로 판단

기본 정책: p > 0.5이면 실패1
p = 0.5이면 성공0

임계값은 모델·자료와 함께
평가 전에 고정할 설정이다.

추정 확률 0.8은 "이 요청이 반드시 80% 확률로 실패한다"는 보장이 아니다.

공식 참고: ROC-AUC / Decision threshold

ROC · 임계값에 따른 발견과 오탐

축의 의미

가로: FPR = FP / (FP + TN)
성공 요청 중 잘못 경고한 비율

세로: TPR = TP / (TP + FN)
실제 실패 중 찾아낸 비율

TPR은 Recall과 같다.

곡선을 읽는 방법

임계값을 바꾸면 점이 이동한다.

좌상단에 가까울수록
해당 임계값에서 발견이 많고 오탐이 적다.

곡선 전체를 하나의 임계값 성능과
동일시하지 않는다.

AUC는 곡선 아래 면적이며, 특정 임계값의 F1이나 확률 보정 정도와 다르다.

공식 참고: ROC-AUC / Metrics and scoring

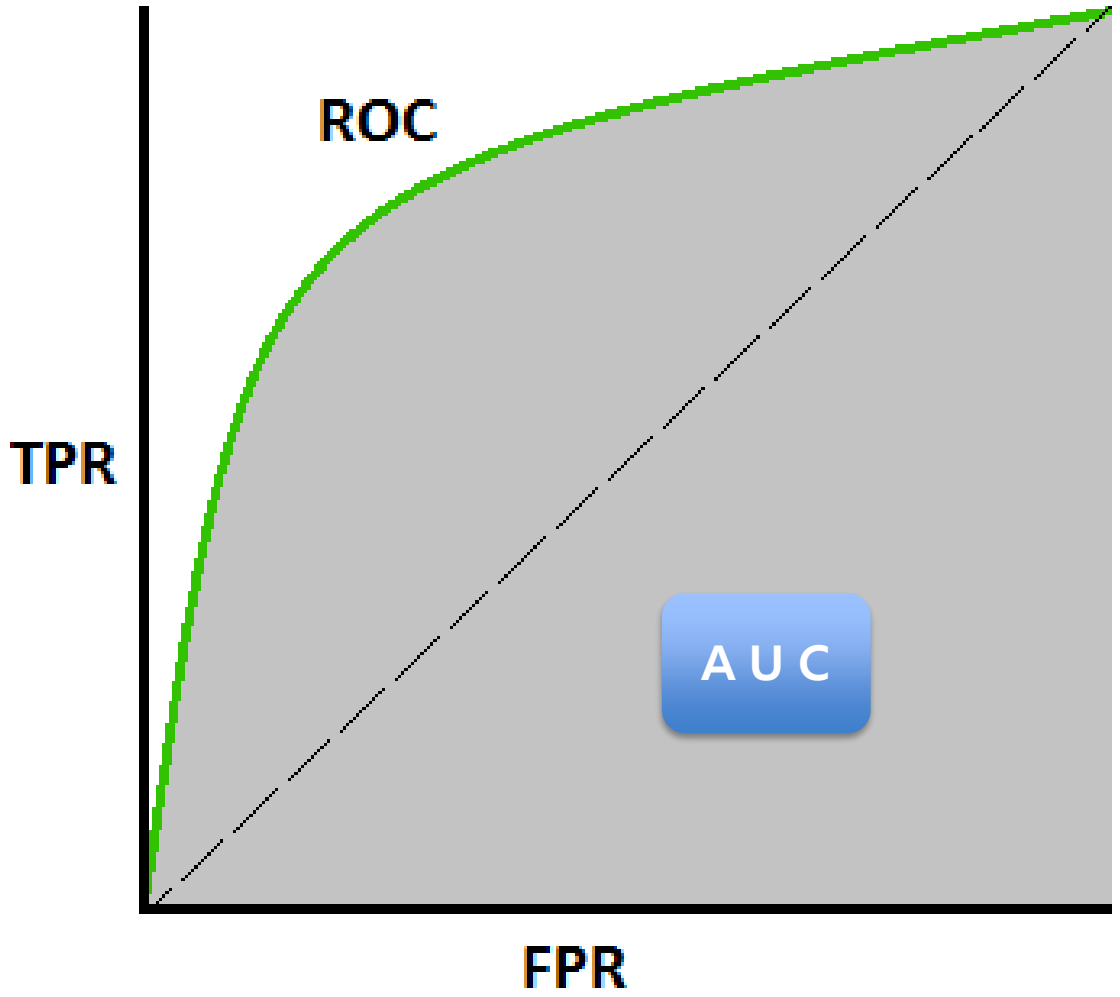
AUC 0.75 · 네 쌍 중 세 쌍의 순위

실제 라벨	점수	비교에서의 의미
성공0	0.10	실패 점수0.35와0.80보다 낮다.
성공0	0.40	실패 점수0.35보다 높고0.80보다 낮다.
실패1	0.35	성공2건 중1건보다 높다.
실패1	0.80	성공2건 모두보다 높다.

양성·음성 2×2=4쌍 중3쌍에서 양성 점수가 높다 → AUC=0.75 (점수 동률 없음).

공식 참고: ROC-AUC

AUC 0.75 · 네 쌍 중 세 쌍의 순위



ROC 좌표와 AUC를 계산한다

```
from sklearn.metrics import roc_curve, roc_auc_score
y = np.array([0, 0, 1, 1])
p = np.array([0.10, 0.40, 0.35, 0.80])
fpr, tpr, thresholds = roc_curve(y, p)
print(fpr)
print(tpr)
print(roc_auc_score(y, p))
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: ROC-AUC

함수 · 매개변수

roc_curve(y, score)
FPR·TPR·임계값 반환

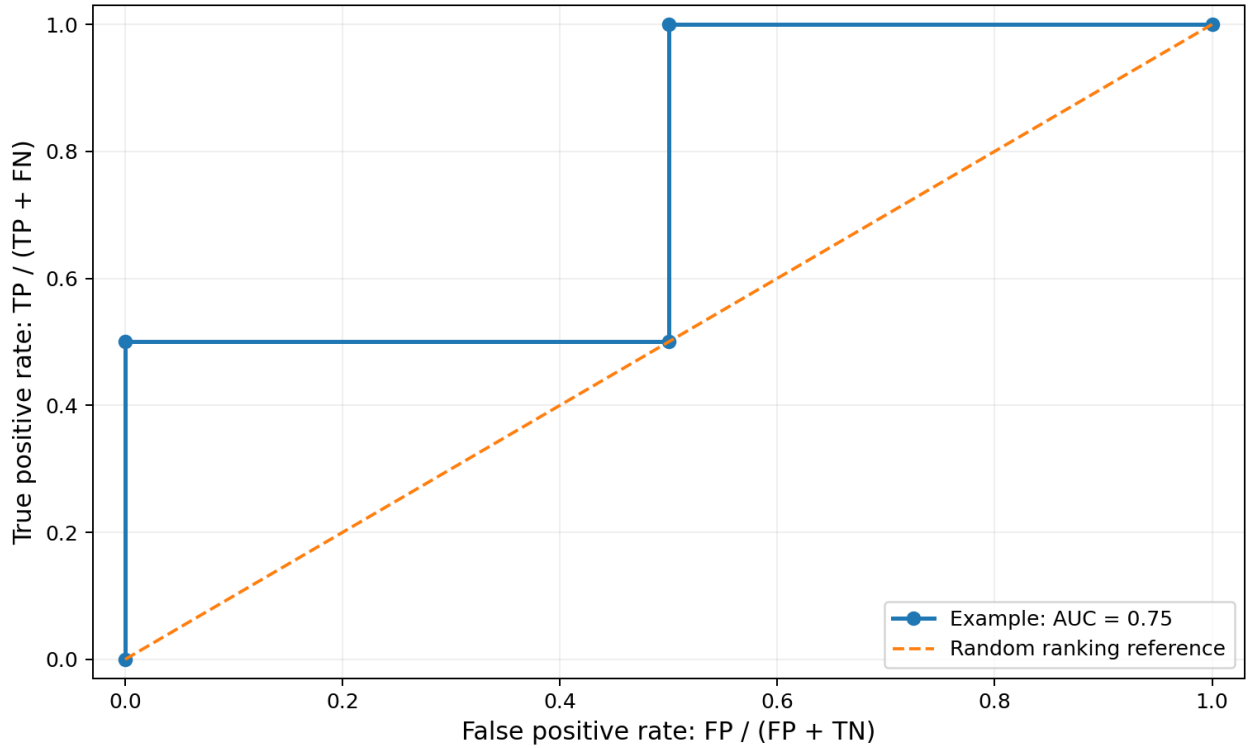
roc_auc_score
양성 점수의 순위 평가
두 실제 클래스가 필요

실행 결과

FPR: 0,0,.5,.5,1
TPR: 0,.5,.5,1,1
AUC: 0.75
순위 비교: 3/4

한로 강민준

같은 점수의 ROC 곡선



곡선과 판단의 차이

실제 실패2건·성공2건

이 자료의 AUC는0.75

임계값0.5에서는
실패2건 중1건만 발견

좋은 순위와 좋은 운영 기준은
별도로 점검한다.

점수4개로 만든 원리 그림이다. 실서비스의 성능 추정치로 사용하지 않는다.

공식 참고: ROC-AUC / Decision threshold

임계값을 바꾸면 오류 비용이 달라진다

```
for t in [0.5, 0.3]:
    pred = (p > t).astype(int)
    tn, fp, fn, tp = confusion_matrix(
        y, pred, labels=[0, 1]).ravel()
    cost = fp + 5 * fn
    print(t, fp, fn, cost)
# 비용 5:1은 설명용 가정, 실제 업무 근거가 아님
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Decision threshold / ROC-AUC

함수 · 매개변수

pred = (p > t)
동일한 점수를 다른 기준으로 판단
cost = FP + 5×FN
설명용 비용 가정
test에서 임계값 탐색 금지

실행 결과

t=0.5: FP0 / FN1
F1=.667, 비용5
t=0.3: FP1 / FN0
F1=.800, 비용1
AUC는 계속0.75

한로 강민준

AUC만 보고 끝내지 않는다

AUC는 순위 평가다. 임계값의 경고 수·Precision·Recall도 확인한다.

불균형 자료에서는 ROC와 함께 실제 양성 수, PR 관점도 살핀다.

한 클래스만 있는 자료에서는 AUC를 계산할 수 없다. 0으로 위장하지 않는다.

확률의 보정과 운영 비용은 별도 문제다. 좋은 AUC만으로 배포를 결정하지 않는다.

실패1의 점수 열·평가 자료·표본 수·임계값을 보고서에 함께 기록한다.

공식 참고: ROC-AUC / Metrics and scoring / Decision threshold

한로 강민준

3분 퀴즈 · AUC와 임계값

Q1. AUC 함수에 0/1 예측 클래스만 넣으면 충분한가?

정답 · 해설 아니다. 실패1의 연속 점수를 사용해야 여러 임계값의 순위를 평가할 수 있다.

Q2. 같은 점수에 임계값만 바꾸면 AUC도 바뀌는가?

정답 · 해설 아니다. 점수 순위가 같으므로 AUC는 같고, FP·FN·F1 등이 바뀐다.

Q3. test F1을 높이는 임계값을 골라도 되는가?

정답 · 해설 안 된다. 그 test를 모델 선택에 사용한 것이므로 최종 독립 평가가 아니게 된다.

공식 참고: ROC-AUC / Decision threshold / Common pitfalls

한로 강민준

실습 02 · ROC와 임계값을 따로 해석한다

기본필수

E03의 4개 점수로 AUC0.75를 계산한다. 양성·음성 네 쌍의 순위를 손으로 확인한다.

응용

E04의 임계값0.5·0.3에서 FP·FN·F1·가정 비용을 비교한다. 같은 AUC인데 운영 결과가 다른 이유를 적는다.

심화

한 클래스만 있는 입력의 AUC 처리 정책 또는 OOF 임계값 선택을 별도 설계한다. 최종test와 분리한다.

완료 기준: 양성 점수 + ROC/AUC + 두 임계값 비교 + test 미사용 근거

한로 강민준

RMSE · 큰 오차를 더 강하게 반영한다

계산 순서

오차 = 실제값 − 예측값

제곱 → 평균 → 제곱근

$RMSE = \sqrt{mean((y - \hat{y})^2)}$

실행시간을 ms로 예측하면
RMSE의 단위도 ms다.

MAE와 함께 해석

MAE: 평균적인 절대오차
RMSE: 큰 오차에 더 민감

평균만 보지 말고 큰 잔차의
실제값·예측값·ID를 확인한다.

회귀 점수와 분류F1을
합쳐서 평균내지 않는다.

높을수록 좋은 scorer 관례 때문에 CV에는 음수 RMSE가 사용된다. 보고할 때 부호를 되돌린다.

공식 참고: root_mean_squared_error / Metrics and scoring

지표에 따라 A와 B의 선택이 달라진다

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import root_mean_squared_error
y = [100, 200, 300, 400]
a = [100, 200, 300, 480]
b = [130, 230, 330, 430]
for name, pred in [('A', a), ('B', b)]:
    print(name, mean_absolute_error(y, pred),
          root_mean_squared_error(y, pred))
```

함수 · 매개변수

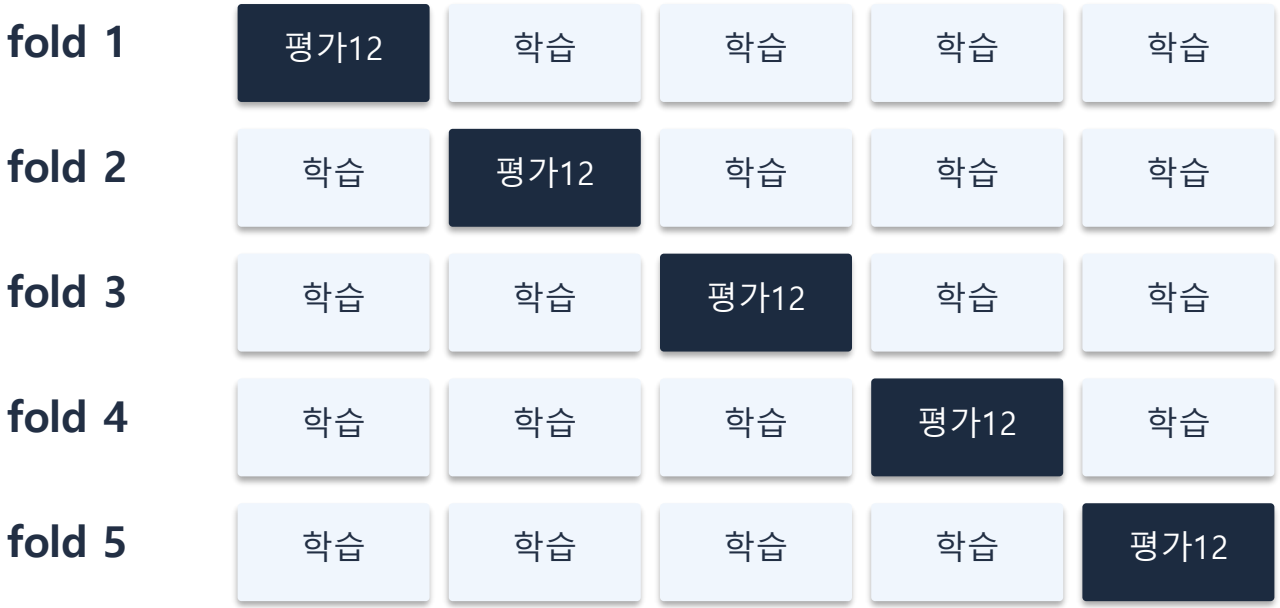
mean_absolute_error
절대오차의 평균
root_mean_squared_error
제곱오차 평균의 제곱근
같은 실제값·같은 단위로 비교

실행 결과

A: MAE20ms
RMSE40ms
B: MAE30ms
RMSE30ms

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록
공식 참고: root_mean_squared_error / Metrics and scoring

5-fold의 test_score는 보관 test가 아니다



각 fold는 새 Pipeline

전처리·모델: 학습48건 fit

평가12건: transform / predict

보관 test20은 참여하지 않음

5개 점수의 평균·변동을 기록

cross_validate의 test_f1 = fold 평가 점수 · 최종 test20 평가와 이름을 구분해서 기록한다.

공식 참고: Cross-validation / cross_validate

한로 강민준

한 번의 CV에서 F1과 AUC를 기록

```
from sklearn.model_selection import cross_validate
scores = cross_validate(
    pipe, X_train, y_train, cv=CLS_FOLDS,
    scoring={'f1': 'f1', 'auc': 'roc_auc'},
    return_train_score=True, error_score='raise',
    n_jobs=1)
print(scores['test_f1'])
print(scores['test_f1'].mean())
# test_f1의 test는 fold 평가12건, 보관test20 아님
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: cross_validate / Cross-validation

함수 · 매개변수

scoring 딕셔너리
같은 fold에 여러 지표
return_train_score
train과 fold 평가 비교
error_score=raise
실패를 조용히 숨기지 않음

실행 결과

fold F1:
.286 .667 .857 .400 .400
평균 F1: .522
표준편차: .209
평균 AUC: .805

한로 강민준

회귀 CV의 음수 부호를 되돌린다

```
scores = cross_validate(
    build(LinearRegression()), X_train, yr_train,
    cv=REG_FOLDS,
    scoring='neg_root_mean_squared_error',
    n_jobs=1, error_score='raise')
rmse = -scores['test_score']
print(rmse)
print(rmse.mean())  # 단위: ms
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: root_mean_squared_error / cross_validate

함수 · 매개변수

neg_root_mean_squared_error
작은 RMSE가 큰 scorer 값
rmse = -test_score
보고값의 부호 복구
KFold: 회귀의 고정5분할

실행 결과

평균 fold RMSE
Linear: 639.83ms
KNN15: 598.85ms
두 모델 모두 train60 내부

한로 강민준

평균·표준편차·합친 점수는 다르다

fold 점수의 요약

예: F1 평균0.522 / 표준편차0.209

분할에 따라 결과가 흔들렸다는
진단 근거로 사용한다.

"평균 ± 표준편차"를 곧바로
95% 신뢰구간이라고 쓰지 않는다.

OOF 예측을 합치면?

각 행을 학습하지 않은 모델로 예측

fold F1 평균과 합친 F1은
분모·비선형성 때문에 다를 수 있다.

같은 크기 두 fold의 RMSE가2·10이면
평균6, 합친 RMSE는 약7.21이다.

평균 낸 대상과 가중 방식을 기록한다. 서로 다른 계산의 숫자를 같은 열에 섞지 않는다.

공식 참고: Cross-validation / cross_val_predict / root_mean_squared_error

자료 구조에 맞는 분할을 사용한다

자료의 구조	검토할 분할	추가 검사
독립 요청·분류	StratifiedKFold	fold별 양성 수·ID 교집합
동일 고객/요청의 반복 행	GroupKFold 등 그룹 분리	같은 그룹이 학습/평가 양쪽에 없는가
시간 순서·미래 예측	TimeSeriesSplit 등 시간 분리	미래 자료가 과거 학습으로 들어오지 않는가
선택한 최고 점수의 낙관성	별도 최종test / 필요 시 nested CV	선택용 자료와 최종 평가의 역할 구분

분할 방법은 점수가 잘 나오는지로 고르지 않고, 실제로 예측할 상황에 맞춰 정한다.

공식 참고: Cross-validation / Common pitfalls

한로 강민준

3분 퀴즈 · CV 점수를 읽는 기준

Q1. CV의 test_f1에 보관한 test20이 들어 있는가?

정답 · 해설 아니다. train60 내부에서 각 fold의 평가12건에 대한 점수다.

Q2. neg_RMSE가 -500과 -700이면 어떤 것이 좋은가?

정답 · 해설 -500이 더 큰 scorer 값이며, 실제 RMSE500ms가700ms보다 작다.

Q3. CV 평균이 조금 높으면 통계적으로 우수한가?

정답 · 해설 그렇게 단정할 수 없다. 표본 수·분산·후보 선택 과정과 별도 평가를 함께 본다.

공식 참고: Cross-validation / cross_validate / root_mean_squared_error

실습 03 · RMSE와 교차검증을 같은 기준으로

기본필수

E05에서 MAE·RMSE의 선택 차이를 설명한다. E06의 fold F1·평균·표준편차를 표로 기록한다.

응용

E07의 회귀 CV를 실행하고 음수 scorer를 RMSE(ms)로 바꾼다. 같은 fold에서 두 모델을 비교한다.

심화

fold 평균 F1과 합친 OOF F1을 비교하거나 그룹/시간 분할을 설계한다. 계산·분할의 차이를 명시한다.

완료 기준: 단위가 있는 회귀 지표 + fold별 점수·평균·편차 + 분할 근거

한로 강민준

특성 설계 · 모델에 보여 줄 표현을 바꾼다

원본 입력에 의미 있는 변환·조합을 만들어 예측에 필요한 정보를 표현한다.

이번 예: 글자 수의 log1p, 글자 수와 허용 도구 수의 비율을 추가한다.

새 특성도 요청 전에 계산 가능해야 한다. 정답·실행 결과를 섞지 않는다.

추가했다고 항상 좋아지지 않는다. 같은 모델·같은 CV로 차이를 검증한다.

가설 → 변환 정의 → 입력/출력 검사 → 고정 조건 비교 → 해석의 한계 순서로 진행한다.

공식 참고: Pipelines and composition / Common pitfalls / FunctionTransformer

함로 강민준

파생 변수 두 개의 의미와 경계

특성	정의	필요한 주의
log_input_chars	log(1 + input_chars)	0에서도0 / 음수 금지 / NaN 유지
chars_per_slot	input_chars / (1 + allowed_tools)	분모0 방지 / 실제 도구당 실행시간 아님
원본4열	수치2열 + 범주2열 그대로 유지	이번 실험은 추가만 하고 삭제하지 않음
금지 파생 변수	elapsed_ms/문자 수, retry_count 등	실행 후 정보에서 만들면 시점 누수

분모의 +1은 명시적인 설계 선택이다. 단위와 의미를 적고 성능 개선은 별도로 확인한다.

공식 참고: NumPy · log1p / Common pitfalls

한로 강민준

행별 파생 변수 · 원본을 바꾸지 않는다

```
def add_features(X):
    out = X.copy()
    out['log_input_chars'] = np.log1p(out['input_chars'])
    out['chars_per_slot'] = (
        out['input_chars'] / (1.0 + out['allowed_tools']))
    return out
# 전체 예제에는 열·비음수·무한대 검사도 포함
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: NumPy · log1p / FunctionTransformer

함수 · 매개변수

np.log1p(x)
log(1+x)를 계산
X.copy()
원본 데이터 수정 방지
NaN은 NaN으로 남겨
학습 내 결측 처리에 전달

실행 결과

chars / tools → log / ratio
0 / 0 → 0 / 0
99 / 2 → 4.605 / 33
999 / 4 → 6.908 / 199.8
NaN → NaN 유지

행별 규칙과 학습해야 할 통계를 구분

행별로 계산 가능한 규칙

log1p(글자 수)
글자 수 / (1 + 허용 도구 수)

한 행의 사전 입력만 사용

그래도 Pipeline에 넣어
학습·추론의 규칙을 일치시킨다.

자료에서 추정하는 변환

중앙값·평균·표준편차
범주 목록·특성 선택·PCA·K-Means

학습 자료 또는 fold 학습 부분에서 fit

정답을 이용하는 인코딩/선택은
더 엄격한 누수 방지 절차가 필요하다.

입력만 쓰는 비지도 변환이라도 평가 자료로 fit하면 전체 모델 평가에 누수가 생길 수 있다.

공식 참고: Common pitfalls / Pipelines and composition / Cross-validation

특성 변환부터 모델까지 한 Pipeline

```
features = FunctionTransformer(
    add_features, validate=False)
pipe = Pipeline([
    ('features', features),
    ('pre', make_pre(engineered=True)),
    ('model', LogisticRegression(C=1, max_iter=1000))])
pipe.fit(X_train, y_train)
print(pipe[:-1].transform(X_train).shape)
print(pipe[:-1].transform(X_valid).shape)
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: FunctionTransformer / Pipelines and composition

함수 · 매개변수

FunctionTransformer
직접 만든 함수를 변환기로 사용
validate=False
DataFrame 입력을 유지
pipe[:-1]
모델 전까지의 변환만 실행

실행 결과

원본: 4열
변환 train: (60,10)
변환 valid: (20,10)
수치4 + 원핫6
NaN·inf 없음

한로 강민준

파생 특성의 효과를 분리해서 비교

```
for name in ['Logistic', 'Logistic_FE']:
    pipe = candidates()[name]
    cv = cross_validate(pipe, X_train, y_train,
        cv=CLS_FOLDS, scoring='f1', n_jobs=1)
    pipe.fit(X_train, y_train)
    print(name, cv['test_score'].mean(),
          f1_score(y_valid, pipe.predict(X_valid)))
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Cross-validation / Pipelines and composition

함수 · 매개변수

고정: Logistic C=1
같은 train60·5-fold
같은 결측·표준화·원핫
변경: 파생 특성2개 추가
valid 결과도 숨기지 않음

실행 결과

기존 → 파생 추가
CV F1: .522 → .551
CV 표준편차: .209 → .190
valid F1: .333 → .182

한로 강민준

중앙값이 각 fold 학습에서 왔는가?

```
cv = cross_validate(pipe, X_train, y_train,
    cv=CLS_FOLDS, scoring='f1', return_estimator=True)
for (fit_idx, _), fitted in zip(
        CLS_FOLDS, cv['estimator']):
    values = add_features(X_train.iloc[fit_idx])
    expected = values[NUM + DERIVED].median()
    num = fitted['pre'].named_transformers_['num']
    actual = num['imputer'].statistics_
    assert np.allclose(expected.to_numpy(), actual)
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: cross_validate / Common pitfalls

함수 · 매개변수

return_estimator=True
fold별 fitted Pipeline 반환
statistics_
학습한 결측 대체값
np.allclose
실수 오차를 허용해 비교

실행 결과

글자 수 중앙값
1309 / 1303 / 1315
1315 / 1338
5개 fold 모두 검사 통과
각 fold 학습48건만 사용

한로 강민준

3분 퀴즈 · 특성 추가와 누수

Q1. 실행시간/글자 수를 실행 전 실패 예측에 넣어도 되는가?

정답 · 해설 안 된다. 실행시간은 예측 시점에 없는 결과 정보다.

Q2. 파생 특성 추가 후 CV가0.029 상승했으니 개선 확정인가?

정답 · 해설 아니다. 변동이 크고 valid는 낮아졌다. 현재 결과와 한계를 모두 보고한다.

Q3. 수치형 결측을 전체 자료 중앙값으로 먼저 채워도 되는가?

정답 · 해설 안 된다. fold별 학습 부분의 중앙값을 사용해야 한다.

공식 참고: Common pitfalls / Cross-validation

실습 04 · 누수 없는 특성 설계와 비교

기본필수

E08의 행별 특성2개를 만들고 원본·행 순서가 유지되는지 확인한다. E09의 train(60,10)·valid(20,10)을 재현한다.

응용

Logistic 기본/파생 추가를 같은5-fold로 비교한다. CV·valid 차이와 변수의 의미·NaN 처리 규칙을 적는다.

심화

E11의 fold별 학습 통계를 자동 검사한다. 정답을 이용하는 특성 선택이 CV 밖에 있을 때의 오류도 설명한다.

완료 기준: 특성 정의 + 변환 전후 표 + 동일 조건 비교 + fold 내 학습 검사

한로 강민준

KNN과 K-Means는 다른 문제를 푼다

KNN · 지도학습

입력과 정답으로 예측한다.

K = 참고할 이웃 수

가까운 학습 사례의 라벨·값을
새 입력의 예측에 사용한다.

K-Means · 비지도학습

정답 없이 비슷한 입력을 묶는다.

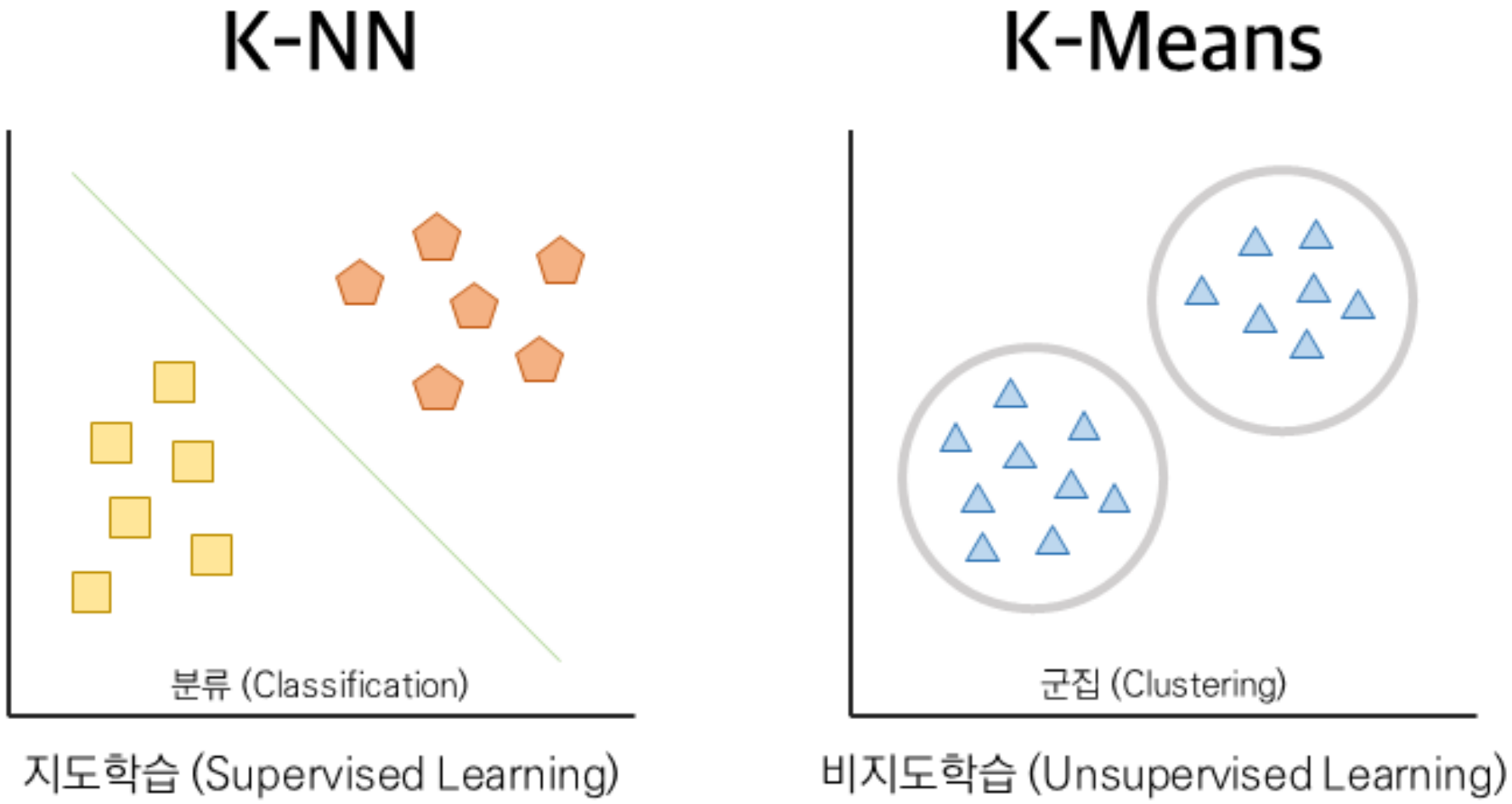
K = 만들 군집의 개수

각 군집의 중심을 학습하고
새 입력을 가까운 중심에 할당한다.

군집0·1·2는 순서나 성공/실패 의미가 없는 이름표다.

공식 참고: Clustering / KMeans

KNN과 K-Means는 다른 문제를 푼다



공식 참고: Clustering / KMeans

K-Means · 할당과 중심 갱신을 반복

초기화

K개 중심을 선택
k-means++

›

할당

각 점을 가장 가까운
중심에 배정

›

갱신

각 군집에 속한
점들의 평균으로 이동

›

반복·종료

할당/중심이 안정되거나
종료 조건에 도달

목적은 중심까지 제곱거리의 합(inertia)을 줄이는 것. 초기값에 따라 해가 달라질 수 있다.

공식 참고: KMeans / Clustering

6개 점을 두 군집으로 묶는다

```
from sklearn.cluster import KMeans
X = np.array([[1,1], [1,2], [2,1],
              [8,8], [8,9], [9,8]], dtype=float)
km = KMeans(n_clusters=2, n_init=10, random_state=42)
labels = km.fit_predict(X)   # y를 전달하지 않는다
print(labels)
print(km.cluster_centers_)
print(km.inertia_)
```

함수 · 매개변수

n_clusters=2
군집 개수
n_init=10
여러 초기화 시도
random_state=42
초기화 재현

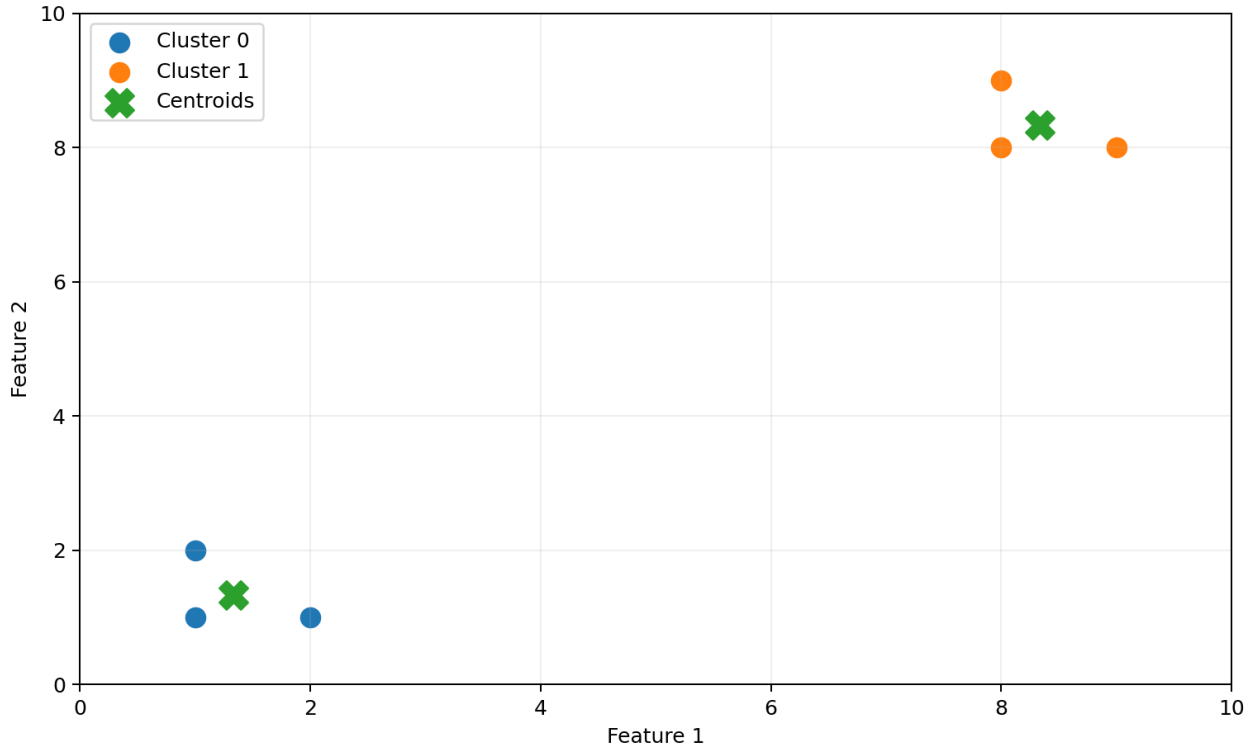
실행 결과

labels: 0,0,0,1,1,1
중심: (1.333,1.333)
(8.333,8.333)
inertia: 2.667

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: KMeans

군집 중심은 소속 점들의 평균



이 그림에서 확인할 것

작은 점: 입력6건
X 표시: 중심2개

라벨0과1을 서로 바꿔도
같은 묶음이다.

정답 없이 만든 묶음에
성공/실패 의미를 붙이지 않는다.

두 축의 단위가 같은 원리 예제. 실제 요청의 글자 수·도구 수에는 스케일 차이가 있다.

공식 참고: Clustering / KMeans

로그 변환 · 결측 대체와 표준화부터

```
cluster_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('cluster', KMeans(n_clusters=3, n_init=10,
                       random_state=42))])
cluster_pipe.fit(X_train[NUM])
labels = cluster_pipe['cluster'].labels_
print(np.bincount(labels))
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: KMeans / Pipelines and composition

함수 · 매개변수

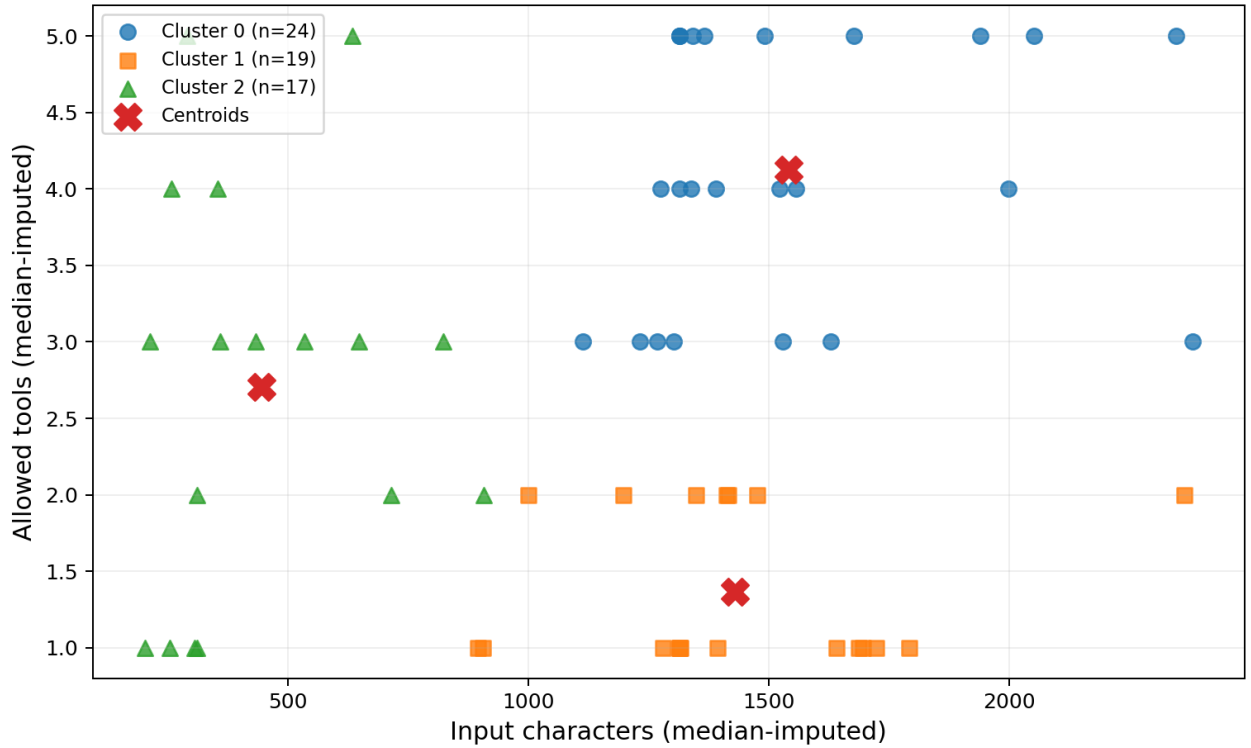
입력: train60 수치2열
SimpleImputer
train 중앙값으로 결측 대체
StandardScaler
거리 계산 전 단위 차이 조정

실행 결과

train 입력: (60,2)
K=3 군집별 개수
24 / 19 / 17
inertia: 40.430
정답 y는 사용하지 않음

한로 강민준

로그 변환의 군집과 대표 특징



원단위 중심으로 읽기

군집0: 약1542자 / 도구4.12
군집1: 약1430자 / 도구1.37
군집2: 약445자 / 도구2.71

도구 수 중심이 소수인 이유는
군집 내부 평균이기 때문이다.

표시: 원단위 / 학습: 표준화

중심은 결측 대체 후 좌표의 평균이다. "실제 사용한 도구 수" 또는 "성공률"로 해석하지 않는다.

공식 참고: KMeans / Clustering

군집 평가 · inertia와 silhouette

inertia · 군집 안의 거리

각 점에서 소속 중심까지
제곱거리의 합

같은 자료·같은 스케일에서 비교

K가 늘면 더 작아지기 쉬우므로
최소값만으로 K를 고르지 않는다.

silhouette · 안과 밖의 거리

같은 군집과의 거리,
다른 군집과의 거리를 비교

범위는 −1~1, 높을수록
현재 거리 공간에서 분리가 좋다.

실제 군집 수가2 이상·표본 수 미만 필요

분리도가 높아도 업무적으로 의미 있는 집단이라는 보장은 없다. 표본 수·형태·해석을 함께 본다.

공식 참고: silhouette_score / KMeans / Silhouette analysis

한로 강민준

동일한 전처리로 K=2·3·4 비교

```
for k in [2, 3, 4]:
    pipe = numeric_cluster(X_train, k)
    Z = pipe[:-1].transform(X_train[NUM])
    km = pipe['cluster']
    score = silhouette_score(Z, km.labels_)
    print(k, km.inertia_, score)
# 같은 입력·전처리에서 비교, 업무 의미도 따로 검토
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: silhouette_score / KMeans

함수 · 매개변수

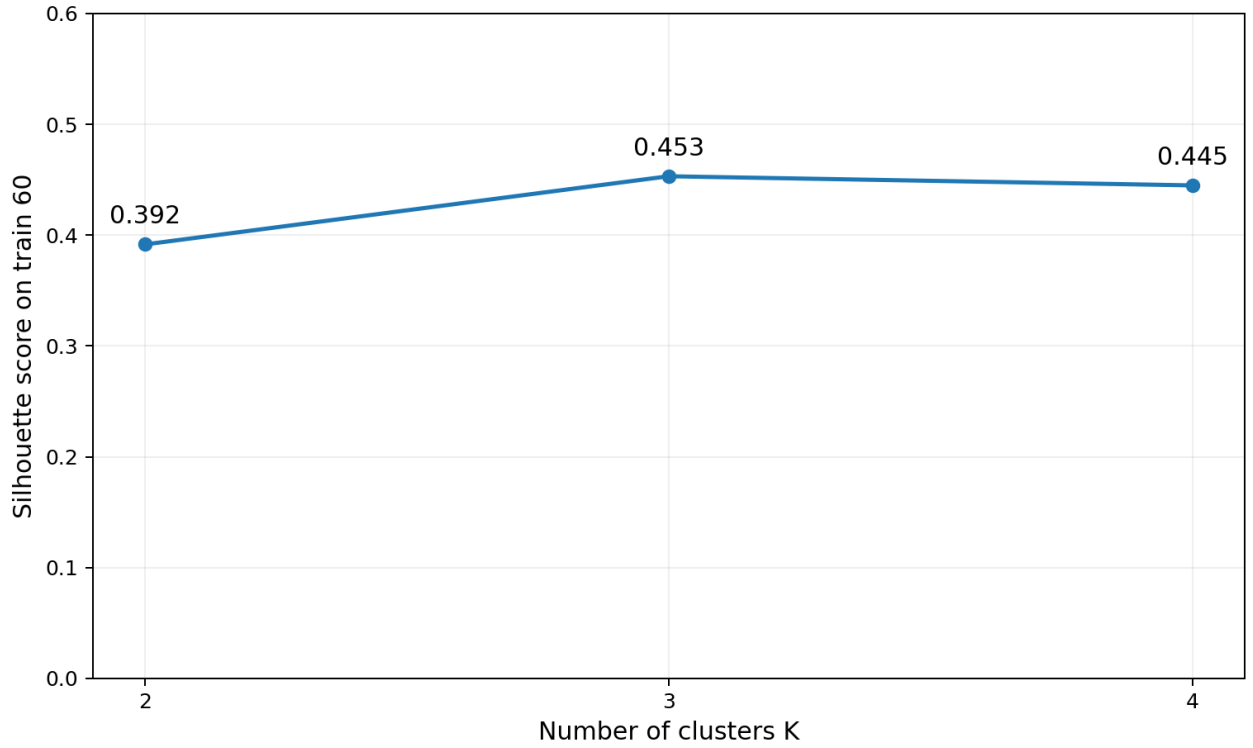
pipe[:-1].transform
같은 표준화 공간으로 변환
silhouette_score(Z, labels)
이 공간에서 분리도 계산
K 후보는 미리 제한

실행 결과

K	inertia	silhouette
2	69.864	.392
3	40.430	.453
4	30.374	.445

함로 강민준

K가 많다고 항상 더 유용하지 않다



이번 자료의 관찰

K=3의 silhouette가 가장 높다.
K=4는 inertia가 더 작다.

한 지표의 값만 보고
항상 군집 수를 늘리지 않는다.

이 수업에서는 K=3으로
개수·중심·입력 의미를 설명한다
.

동일 train60·동일 전처리의 교육용 결과다. 새로운 자료에서도 K=3이 최선이라는 뜻은 아니다.

공식 참고: silhouette_score / Silhouette analysis

새 요청에는 predict로 군집을 할당

```
new = pd.DataFrame({'input_chars': [200., 2200.],
                    'allowed_tools': [1., 5.]})
before = cluster_pipe['cluster'].cluster_centers_.copy()
print(cluster_pipe.predict(new))
print(np.array_equal(before,
    cluster_pipe['cluster'].cluster_centers_))
# 새 요청에서 fit_predict를 다시 호출하지 않는다
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: KMeans

함수 · 매개변수

predict(new)
기존 중심에 새 입력 할당
fit_predict(new)는 금지
중심을 새 자료로 다시 학습함
중심 불변도 검사

실행 결과

새 요청 2건 → [2,0]
200자/도구1 → 군집2
2200자/도구5 → 군집0
중심 불변: True

한로 강민준

3분 퀴즈 · 군집의 의미

Q1. 군집2는 군집0보다 위험한 요청인가?

정답 · 해설 아니다. 군집 번호에는 크기·순서·성공/실패 의미가 없다.

Q2. K-Means는 정답을 안 쓰므로 전체100건으로 fit해도 될까?

정답 · 해설 예측 모델 평가와 연결하는 변환이라면 안 된다. 이 수업은 train60만 사용한다.

Q3. silhouette가 높으면 실제 업무에서도 유용한가?

정답 · 해설 반드시 그렇지 않다. 거리 공간의 분리도와 업무적 의미는 별도로 검토한다.

공식 참고: Clustering / silhouette_score / Common pitfalls

한로 강민준

실습 05 · 군집의 수치와 의미를 설명한다

기본필수

E13의 train 수치2열로 K=3을 실행해 개수24·19·17과 중심을 기록한다. 군집 번호가 정답이 아닌 이유를 설명한다.

응용

K=2·3·4의 inertia·silhouette를 비교하고 표준화 공간을 확인한다. 새 요청에 predict를 적용한다.

심화

군집 번호를 바꿔도 같은 묶음임을 검사하거나, K-Means를 특성 변환으로 쓸 때 CV 안에 넣는 구조를 설계한다.

완료 기준: 군집 개수·중심 + 지표/단위 + 업무 해석과 한계 + test 미사용

한로 강민준

test를 열기 전에 선택 계획을 확정한다

무엇: 실패1 예측 / 입력4열 / 파생 특성 규칙 / 후보 목록을 기록한다.

어떻게: train60의 동일5-fold 평균F1 최대 / 동점 규칙을 미리 정한다.

판단: 임계값0.5 고정 / p>0.5면 실패 / 보고 지표·표본 수를 명시한다.

그다음: 설정 저장 → 새 Pipeline을80건으로 재학습 → test20 평가.

valid는 이미 개발에 사용한 자료다. 최종 독립 평가처럼 다시 포장하지 않는다.

공식 참고: Cross-validation / Common pitfalls / Decision threshold

사전 선언한 7개 후보의 개발 결과

모델	CV F1	fold 표준편차	valid F1
Dummy	0.000	0.000	0.000
Logistic	0.522	0.209	0.333
KNN3	0.336	0.176	0.545
Tree	0.542	0.124	0.333
Forest	0.280	0.254	0.222
XGBoost	0.500	0.123	0.182
Logistic + 파생2개	0.551	0.190	0.182

CV F1 규칙으로 Logistic_FE 선택. Tree와의 약0.009 차이를 확실한 우월성이라고 주장하지 않는다.

공식 참고: Cross-validation

한로 강민준

선택 함수는 test를 입력받지 않는다

```
table, fold_scores = compare_candidates(
    X_train, y_train, X_valid, y_valid, CLS_FOLDS)
selected = select_name(table)
print(table[['model', 'CV_F1', 'CV_std', 'valid_F1']])
print('선택:', selected)
# CV F1 최대; 완전 동점이면 사전 후보 순서
# valid는 진단용, test는 아직 접근하지 않는다
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Cross-validation / Common pitfalls

함수 · 매개변수

compare_candidates
train CV와 valid 진단만 생성
select_name
CV F1 최대 규칙
반올림 전 값으로 비교

실행 결과

선택: Logistic_FE
CV F1: .551
CV 표준편차: .190
valid F1: .182
test: 아직 평가하지 않음

한로 강민준

선택과 재학습과 평가는 별도 단계

선택

train60 내부 CV
valid는 진단 기록

❯

동결

설정·특성·임계값
evaluation_lock.json

❯

재학습

새 Pipeline 생성
train+valid80 fit

❯

최종 보고

test20
transform/predict
지표·예측·한계 저장

80건 재학습 후 과거 valid 성능은 독립 검증 점수가 아니다. test 점수로 다시 후보를 바꾸지 않는다.

공식 참고: Cross-validation / Pipelines and composition / Common pitfalls

한로 강민준

확정한 설정을 저장하고 개발80건 재학습

```
protocol = protocol_for(selected)
Path('evaluation_lock.json').write_text(
    json.dumps(protocol, ensure_ascii=False, indent=2),
    encoding='utf-8')
X_dev = pd.concat([X_train, X_valid])
y_dev = pd.concat([y_train, y_valid])
final = clone(candidates()[selected]).fit(X_dev, y_dev)
# 최종 설정 확정 이후에만 개발80건으로 새로 학습
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Pipelines and composition / Cross-validation

함수 · 매개변수

clone(...)
학습 상태 없는 새 Pipeline
pd.concat
고정 순서로 train+valid 결합
이후 fit의 허용 자료: 80건

실행 결과

재학습80 / test20
수치 중앙값:
1278 / 3
log: 7.154
ratio: 309.8

함로 강민준

동결된 모델을 test20에 적용한다

```
# protocol 저장 + 개발80건 재학습이 끝난 후 실행
X_test = data.loc[TEST_IDS, FEATURES]
y_test = data.loc[TEST_IDS, 'failed']
p = failure_scores(final, X_test)
pred = (p > protocol['threshold']).astype(int)
report = classification_metrics(y_test, pred, p)
print(report)
# test 결과를 본 뒤 모델·특성·임계값을 바꾸지 않음
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: ROC-AUC / f1_score / Common pitfalls

함수 · 매개변수

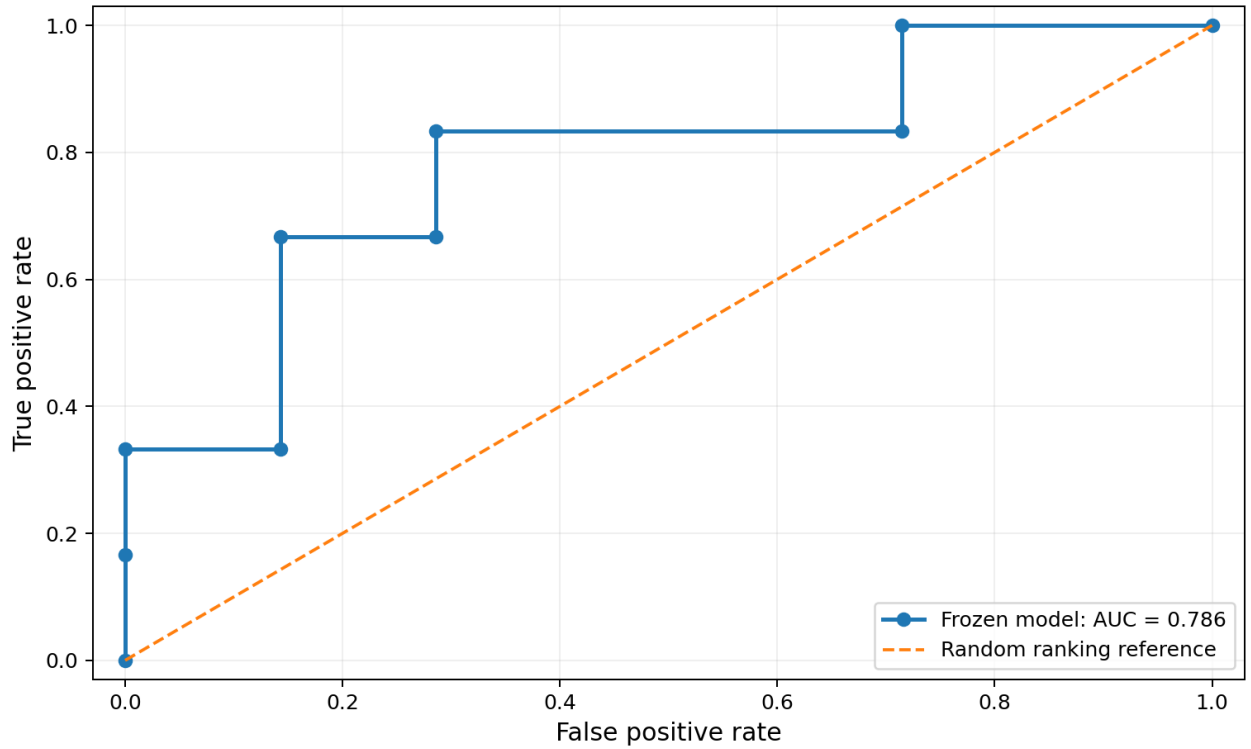
failure_scores
classes_에서 실패1의 열 선택
보고: 라벨 지표 + 점수 AUC
선택 후 test로 fit·
임계값 수정은 하지 않음

실행 결과

Acc .750 / P .600
Recall .500 / F1 .545
AUC .786
TN12 FP2 FN3 TP3
실패 발견: 3/6

한로 강민준

최종 점수는 한계와 함께 보고한다



이 모델에서 남은 문제

최종test는20건뿐이다.
실제 실패6건 중3건을 놓쳤다.

AUC0.786이라고
실패78.6%를 찾은 것은 아니다.

이 결과만으로 실서비스
배포 가능성을 보장하지 않는다.

고정된 교육용 버전의 결과다. 이미 결과를 본 뒤 선택을 바꾸면 새 독립 평가 자료가 필요하다.

공식 참고: ROC-AUC / Cross-validation

3분 퀴즈 · 최종 평가의 경계

Q1. 재학습80건에서 중앙값이 바뀌면 오류인가?

정답 · 해설 아니다. 설정 확정 뒤 허용한 개발80건으로 새로 fit한 결과인지 확인한다.

Q2. test F1이 낮으니 KNN으로 바꾸어 다시 평가하면?

정답 · 해설 test를 선택에 사용한 것이다. 기존 점수는 탐색 결과로 남기고 새 독립 평가가 필요하다.

Q3. 동일 모델·동일 입력으로 결과 파일만 재생성해도 누수인가?

정답 · 해설 아니다. 고정 버전의 재현 확인과, test에 맞춘 반복 선택을 구분한다.

공식 참고: Cross-validation / Common pitfalls

한로 강민준

실습 06 · 평가 계획을 잠그고 최종 결과를 남긴다

기본필수

후보·주 지표·특성·임계값0.5를 기록한다. E16~E18의 순서로 개발80건 재학습 후 test20을 평가한다.

응용

CV·valid·test의 자료와 역할을 구분해 표로 쓴다. 실제값·확률·예측값을 저장하고 오류3건을 설명한다.

심화

동결 뒤 설정 변경을 감지하거나 입력·분할·모델 파일 해시를 검사한다. 임계값 탐색은 별도 사전 계획으로 분리한다.

완료 기준: 평가 계획 파일 + 최종 지표/예측 + 실패 발견3/6 해석 + 재선택 금지 기록

한로 강민준

최종 프로젝트 · 기능보다 재현을 완성

평가: 양성1의 Accuracy·Precision·Recall·F1·AUC와 혼동행렬을 기록한다.

실험: 파생 특성 비교·fold 내부 전처리·CV 결과·확정 설정을 남긴다.

탐색: train 수치2열의 K-Means 개수·중심·분리도·의미를 별도로 보고한다.

재현: 원본·분할표·코드·환경·결과·Git 이력으로 다시 실행 가능하게 만든다.

실습07의25분 + 실습08의35분 = 60분 통합. 이미 만든 함수와 제공 골격을 연결한다.

한로 강민준

실습에서 구성할 VS Code 프로젝트

```
day8_final/
  main.py
  feature_ops.py
  preprocessing.py
  evaluate.py
  checks.py
  data/requests.csv
  data/split_manifest.csv
  data/split_order.csv
  outputs/
  reports/final_report.md
  README.md
  requirements-lock.txt
```

책임을 나누는 기준

feature_ops: 행별 특성 규칙
preprocessing: 열별 전처리
evaluate: 지표·CV·최종 평가
checks: 입력·ID·결과 검사
main: 단계 순서와 파일 저장

실행 위치와 관계없이
프로젝트 기준 경로를 사용한다.

한 번의 흐름으로 결과를 저장하고 재확인

```
# main.py에서 각 단계의 함수를 순서대로 연결
comparison, folds = compare_candidates(
    X_train, y_train, X_valid, y_valid, CLS_FOLDS)
protocol = protocol_for(select_name(comparison))
# 1. CV 결과와 protocol 저장
final = refit_frozen(protocol, X_train, y_train,
                     X_valid, y_valid)
report, predictions = evaluate_frozen(final, protocol, data)
# 2. 고정 버전 test 결과 저장 → 다시 읽어 검증
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Pipelines and composition / Cross-validation

함수 · 매개변수

선택 → 설정 저장
재학습 → test 평가 → 결과 저장
pd.read_csv
ID·행 수·지표 재확인
같은 test를 재탐색하지 않음

실행 결과

핵심 결과 파일5개
예측 행 수20 / ID일치
F1 재계산: .545
lock: Logistic_FE

한로 강민준

모델뿐 아니라 변환 함수도 함께 보관

```
# feature_ops.py에 정의한 이름 있는 함수를 import
from feature_ops import add_features
# final의 FunctionTransformer도 위 함수를 사용
joblib.dump(final, 'final_pipeline.joblib')
restored = joblib.load('final_pipeline.joblib')
assert np.allclose(final.predict_proba(X_check),
                   restored.predict_proba(X_check))
# 본인이 만든 파일만 load. 코드·버전·임계값도 함께 보관
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Model persistence / FunctionTransformer

함수 · 매개변수

joblib.dump / load
학습 Pipeline 저장·복원
이름 있는 함수를 모듈에 정의
같은 환경·코드로 재현
신뢰할 수 없는 파일 load 금지

실행 결과

새 프로세스 복원: True
함수 모듈: feature_ops
확률 예측 동일
비교 자료는 재현 확인용

한로 강민준

새 범주와 열 누락은 다른 상황이다

```
checked = check_input(new_requests)
num = fitted['pre'].named_transformers_['num']
before = num['imputer'].statistics_.copy()
p = failure_scores(fitted, checked)
after = num['imputer'].statistics_
assert np.array_equal(before, after)
check_input(new_requests.drop(columns='priority'))
# 마지막 줄: ValueError가 기대되는 실패 시험
```

전체 실행 코드·전제·전체 출력은 발표자 노트에 수록

공식 참고: Common pitfalls / Pipelines and composition

함수 · 매개변수

미지 범주
원핫ignore로 기존 열 수 유지
필수 열 누락
명확한 오류로 중단
검사 후 예측·통계 불변 확인

실행 결과

새 입력 shape: (2,10)
유한 확률: True
통계 불변: True
priority 누락: ValueError

한로 강민준

출력 파일에 역할과 단위를 남긴다

산출물	기록할 내용
cv_comparison.csv / cv_scores.csv	모델·특성·fold 점수·평균·표준편차·CV 지표
feature_comparison.csv	기존/파생 추가의 동일 조건 비교·해석
cluster_profile.csv / cluster_scores.csv	군집 개수·중심·K·inertia·silhouette·학습 자료
evaluation_lock.json	선택 모델·설정·특성 규칙·임계값·ID·평가 정책
test_metrics.json / test_predictions.csv	최종 지표·실제값·확률·예측·ID·표본 수
README / reports/final_report.md	실행 명령·환경·근거·한계·Git 변경 이력

응용: final_pipeline.joblib + feature_ops.py + 환경 기록 + 새 프로세스 복원 검사

한로 강민준

정상·경계·실패 시험의 기대값을 먼저 적는다

상황	기대 결과	판정 근거
정상 입력20건	확률·예측 각각20개	ID·행 순서 유지, 지표 재계산 일치
수치 결측·새 범주	원래 경로로 변환·예측	10열 유지·NaN/inf 없음·통계 불변
priority 필수 열 누락	예측 전에 ValueError	임의 값으로 조용히 보완하지 않음
한 클래스뿐인 평가	AUC 계산 불가 표시	None/null + 실제 양성 수 기록
동결 뒤 설정 변경	새 버전 또는 평가 중단	기존 최종 성능과 구분하여 기록

코드가 멈추지 않았다는 것과, 요구한 정책대로 실행됐다는 것은 다르다.

공식 참고: ROC-AUC / Common pitfalls

보고서는 점수보다 판단 과정을 설명한다

근거를 포함한 설명

선택: train60 CV F1 규칙을 따랐다.

파생 추가: CV는0.551, valid는0.182였다.

최종test: F1 0.545, 실패3/6 발견

군집: 정답 없는 입력 구조를 요약했다.

반드시 적을 한계

test20건·실패6건의 소표본

여러 후보·여러 날의 개발 조회

합성 데이터, 실제 서비스 검증 아님

좋은 ROC-AUC가 운영 비용이나
확률 보정을 보장하지는 않는다.

지표·자료·임계값을 함께 쓴다. CV 점수와 최종 test 점수를 같은 성능이라고 합치지 않는다.

공식 참고: Cross-validation / ROC-AUC

한로 강민준

실습 07 · 최종 모델링 프로젝트 통합

기본필수

앞 실습을 연결해 CV·특성 비교·군집 요약·동결 설정·test 결과를 생성한다. 제공된 골격을 사용해도 된다.

응용

코드를 모듈로 나누고 Pipeline을 저장·복원한다. 짝이 README만 보고 같은 결과를 다시 만들도록 한다.

심화

fold 학습 통계·새 범주·열 누락·동결 설정 변경 중 두 가지 이상을 자동 검사하고 실패 이유를 남긴다.

완료 기준: 실행 초안 + 평가/특성/군집 결과 + 설정 파일 + test 기록

과목 실습평가 · 성능보다 올바른 실험

개인 설명: 주 지표·분모·양성 라벨·선택 이유를 본인의 말로 설명한다.

실행 시연: 원본·고정 분할에서 결과를 다시 생성하고 ID·수치를 확인한다.

검증 증거: 전처리 통계·최종 평가 정책·정상/실패 입력 기록을 제시한다.

결과 제출: 코드·보고서·환경·Git 이력을 한 프로젝트에 정리한다.

점수 절대값·복잡한 모델 사용·심화 수행 여부로 가점을 강제하지 않는다.

한로 강민준

과목 실습평가 기준 · 공통 100점

평가 항목	배점	충족 기준
데이터·평가 계획	20	고정 ID·예측 시점·누수 방지·test 정책
지표 계산·해석	20	분류 지표/AUC·회귀RMSE·분모/단위·오류 해석
특성·Pipeline·CV	25	특성 정의·fold 내부 fit·같은 조건 비교
K-Means 해석	10	수치 전처리·개수/중심·silhouette·의미의 한계
재현·검증·협업	15	재실행·정상/실패 검사·README·환경·Git 이력
보고서·개인 설명	10	선택 근거·CV/test 구분·소표본 한계

기본필수만으로100점 기준을 충족할 수 있다. 응용·심화는 선택 확장이다.

한로 강민준

마지막 35분 · 재실행부터 제출까지

10분

결과 재생성
고정 버전·입력 확인

›

10분

짝의 실행 재현
테스트 기록 확인

›

10분

보고서·개인 설명
근거와 한계 정리

›

5분

파일·경로 점검
Git 기록·제출

동일한 최종 버전의 재현은 허용한다. test 점수를 보고 설정을 바꾸는 재탐색은 하지 않는다.

실습 08 · 최종 검증·보고서·시연

기본필수

동일 프로젝트를 다시 실행해 결과를 대조한다. 지표·파생 특성·군집·test 역할을 설명하고 최종 보고서를 제출한다.

응용

저장 모델을 새 프로세스에서 복원하고 확률·예측을 비교한다. 짝의 재현 결과와 수정 내역을 남긴다.

심화

선택/검증 편향·임계값 정책·배포 전 확인할 조건을 설명한다. 다음 자료에서 사용할 새 평가 계획을 제안한다.

완료 기준: 코드 + 지표/특성/군집 표 + 동결/test 기록 + 보고서 + Git 이력

마무리 3분 확인 퀴즈

Q1. AUC0.786이면 실패78.6%를 발견했다는 뜻인가?

정답 · 해설 아니다. AUC는 순위 평가이며, 실제 실패 발견 비율은 Recall이다.

Q2. 파생 특성과 K-Means도 전처리 학습 범위를 지켜야 하는가?

정답 · 해설 그렇다. 통계를 학습하는 변환은 학습 부분 안에서만 fit한다.

Q3. 최종test를 본 뒤 개선하려면 어떻게 기록해야 하는가?

정답 · 해설 기존 결과를 보존하고 개발이 재개되었음을 밝힌 뒤 새 독립 평가를 계획한다.

공식 참고: ROC-AUC / Cross-validation / Common pitfalls

만족도 조사 · 학습평가와 분리한다

이해도: 지표·CV·특성 설계·군집·최종 평가 중 어려웠던 내용을 고른다.

난이도·속도: 기본필수의 부담과 응용·심화의 적절성을 각각 답한다.

도움이 된 활동: 설명·코드 시연·페어 실습·리뷰 중 선택한다.

다음 수업 지원: GPU 환경·오류 해결·추가 설명이 필요한 부분을 적는다.

익명 의견은 성적과 분리한다. 필요하다면 별도의 실명 지원 요청 창구를 둔다.

한로 강민준

오늘의 핵심 정리

평가: 지표의 분모·양성·단위를 알고, 같은 자료와 조건으로 비교한다.

설계: 파생 변수와 군집의 의미를 정의하고 학습 범위·누수를 검사한다.

재현: 선택을 확정한 뒤 최종test를 평가하고 코드·환경·근거를 보관한다.

설명할 수 있어야 할 것: 이 모델을 왜 골랐는가, 어떤 오류가 남았는가, 다시 실행할 수 있는가?

한로 강민준

다음 수업 · 모델 평가의 원칙은 계속된다

일자	다음 학습 내용	오늘과의 연결
10/29 (목)	GPU 확인·PyTorch·Tensor·ANN 순전파·손실함수·역전파	학습/추론·손실/평가 구분
10/30 (금)	학습률·Batch·Epoch Dropout·BatchNorm·Early Stopping	과적합·train/valid 변화 해석
11/02 (월)	CNN·이미지 분류·전이학습 CPU/GPU 실행 비교	동일한 분할·지표·재현 기록 유지

GPU 설치는 다음 수업의 호환성 확인 후 진행한다. 오늘은 임의 CUDA·PyTorch 업데이트를 하지 않는다.

공식 참고: PyTorch · Quickstart

REFERENCE

공식 참고자료 · 평가지표와 검증

- sklearn · Metrics and scoring
- sklearn · f1_score
- sklearn · confusion_matrix
- sklearn · ROC-AUC
- sklearn · precision_score
- sklearn · Cross-validation
- sklearn · recall_score
- sklearn · cross_validate

각 항목 클릭 · 전체 주소와 세부 출처는 발표자 노트에 수록

한로 강민준

REFERENCE

공식 참고자료 · 설계·군집·재현

sklearn · FunctionTransformer

sklearn · silhouette_score

sklearn · Pipelines and composition

sklearn · Clustering

NumPy · log1p

sklearn · Decision threshold

sklearn · KMeans

sklearn · Model persistence

각 항목 클릭 · 전체 주소와 세부 출처는 발표자 노트에 수록

한로 강민준